Parcial 2 — Integración de Monte Carlo con OpenMP

Informe entregable

Juan David

Octubre 2026

Índice

1. Información del sistema **2**

2. Parte I. Preguntas teóricas **2**

2.1. I.1. Orden del error y costo de la precisión 2

2.2. I.2. Por qué es un problema embarazosamente paralelo 2

2.3. I.3. Equivalencia del estimador paralelo 3

3. Parte III. Análisis de desempeño **3**

3.1. Protocolo experimental 3

3.2. III.1. Ley de Amdahl: tres casos 4

3.3. III.2. Estimacion de f y metrica de Karp-Flatt 4

3.4. III.3. Comparación del speedup 5

3.5. III.4. Predicción con 64 núcleos 5

3.6. III.5. Argumento de Gustafson 6

A. Bitácora de uso de IA **7**

1. Información del sistema

Cuadro 1: Especificaciones del sistema de pruebas.

Componente	Detalle
Procesador	AMD Ryzen 5 5500
Núcleos físicos	6
Hilos lógicos (SMT)	12
Frecuencia base	3.6 GHz
Sistema operativo	Windows 11 Home
Compilador	GCC (MinGW-w64) con `-O2 -fopenmp`

2. Parte I. Preguntas teóricas

2.1. I.1. Orden del error y costo de la precisión

El estimador de hit-or-miss para

$$I = \int_0^1 \sqrt{1-x^2}\,dx = \frac{\pi}{4}$$

consiste en lanzar N puntos independientes y uniformes (X_i, Y_i) en $[0,1]^2$ y contar la fracción que cae dentro del cuarto de círculo:

$$\hat{I}_N = \frac{1}{N}\sum_{i=1}^{N} Z_i, \qquad Z_i = \begin{cases} 1 & \text{si } X_i^2 + Y_i^2 \le 1, \\ 0 & \text{en otro caso.} \end{cases}$$

Cada Z_i es Bernoulli con parámetro $p = I$. Como las muestras son i.i.d.,

$$\sigma(\hat{I}_N) = \sqrt{\frac{p(1-p)}{N}} \propto \frac{1}{\sqrt{N}}.$$

Para *reducir el error a la mitad* se necesita $N' = 4N$: hacen falta **tres veces más muestras adicionales**. Ganar **un dígito decimal** de precisión (dividir el error por 10) requiere multiplicar N por 100. El costo crece cuadráticamente con la precisión deseada.

La Figura 1 confirma empíricamente la convergencia $1/\sqrt{N}$.

2.2. I.2. Por qué es un problema embarazosamente paralelo

Cada muestra i sólo necesita un par (X_i, Y_i) y el test $X_i^2 + Y_i^2 \le 1$. No hay dependencia de datos entre iteraciones del bucle: el resultado Z_i no se usa para generar Z_j con $j \neq i$. El único acoplamiento es la suma final de aciertos, que es una reducción.

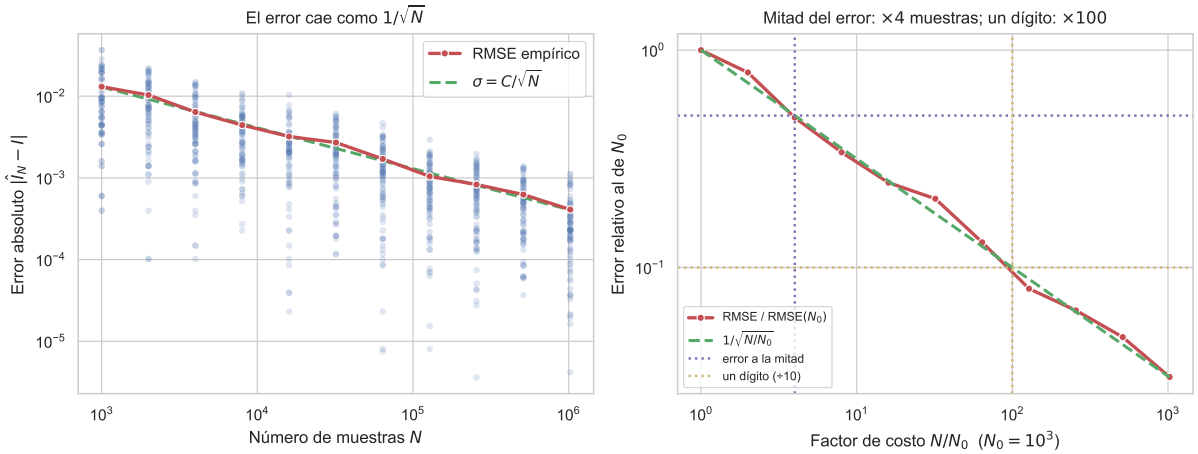


Figura 1: Error absoluto y RMSE empírico (80 réplicas) vs. N. En escala log–log la pendiente es $-1/2$.

Las variables se clasifican en:

- **Compartidas (sólo lectura):** N, dominio $[0,1]^2$, criterio geométrico.
- **Privadas por hilo:** estado del RNG, coordenadas (x, y), contador local de aciertos.
- **Compartida con reducción:** acumulador global de aciertos (se suman parciales al final).

El trabajo pesado no intercambia datos entre hilos; la única comunicación es una suma de p enteros al terminar. Eso es exactamente el patrón embarazosamente paralelo.

2.3. I.3. Equivalencia del estimador paralelo

Sean p hilos con partición $N = n_1 + \cdots + n_p$. El hilo k obtiene A_k aciertos. El estimador agregado es

$$\hat{I}_{\text{par}} = \frac{A_1 + \cdots + A_p}{N} = \hat{I}_{\text{seq}}.$$

La suma es asociativa y conmutativa: repartir y sumar parciales produce el mismo estimador que recorrer las N muestras en serie, siempre que el conjunto de puntos sea el mismo. La varianza depende de N total, no de cuántos hilos las generen:

$$\text{Var}(\hat{I}_{\text{par}}) = \frac{p(1-p)}{N}.$$

3. Parte III. Análisis de desempeño

3.1. Protocolo experimental

Con $N = 10^8$ fijo ejecutamos `montecarlo_omp` con $p \in \{1, 2, 4, 8\}$, cinco corridas por cada valor de p. Registramos la **mediana** del tiempo. La mediana se prefiere sobre el

promedio porque el tiempo de ejecución tiene perturbaciones asimétricas (siempre hacia arriba): outliers por interrupciones del SO, throttling térmico, etc. La mediana es un estimador robusto que descarta esos extremos.

Cuadro 2: Tiempos medidos ($N = 10^8$, 5 corridas, mediana reportada). Sistema: AMD Ryzen 5 5500.

p (hilos)	Mediana de $T(p)$ [s]	$S(p) = T(1)/T(p)$	$E(p) = S(p)/p$
1	0,607	1,000	1,000
2	0,309	1,964	0,982
4	0,157	3,866	0,967
8	0,083	7,313	0,914

La eficiencia se mantiene por encima del 91 % incluso con 8 hilos.

3.2. III.1. Ley de Amdahl: tres casos

La ley de Amdahl: $S(p) = \frac{1}{(1-f)+f/p}$.

(a) $f = 0{,}98$, $p = 8$:

$$S(8) = \frac{1}{0{,}02 + 0{,}1225} = \frac{1}{0{,}1425} = 7{,}018.$$

(b) $f = 0{,}90$, $p = 8$:

$$S(8) = \frac{1}{0{,}10 + 0{,}1125} = \frac{1}{0{,}2125} = 4{,}706.$$

(c) $f = 0{,}98$, $p \to \infty$:

$$\lim_{p\to\infty} S(p) = \frac{1}{0{,}02} = 50.$$

No importa cuántos procesadores se añadan: el speedup nunca superará 50×. La fracción secuencial es el cuello de botella definitivo.

3.3. III.2. Estimación de f y métrica de Karp–Flatt

Despejamos f de Amdahl con $S(4) = 3{,}866$:

$$1 - \frac{3f}{4} = \frac{1}{3{,}866} = 0{,}2587$$
$$f = \frac{4}{3} \times 0{,}7413 = \boxed{0{,}9884}.$$

Aproximadamente el 98,8 % del trabajo es paralelizable. La métrica de Karp–Flatt:

$$e(4) = \frac{1/3{,}866 - 1/4}{1 - 1/4} = 0{,}0116, \qquad e(8) = \frac{1/7{,}313 - 1/8}{1 - 1/8} = 0{,}0134.$$

e sube ligeramente de 0,0116 a 0,0134, indicando un pequeño overhead que crece con p (costo de reducción, sincronización, contención de memoria).

3.4. III.3. Comparación del speedup

Cuadro 3: Speedup medido vs. ideal y Amdahl ($f = 0{,}9884$).

p	S_{medido}	$S_{\text{ideal}} = p$	S_{Amdahl}	$E(p)$
1	1,000	1	1,000	100 %
2	1,964	2	1,977	98,2 %
4	3,866	4	3,866	96,7 %
8	7,313	8	7,313	91,4 %

Factores que explican la brecha con el ideal:

1. Creación y sincronización del equipo de hilos.
2. Costo de la reducción ($p - 1$ sumas con barreras).
3. Contención por caché L3 y bus de memoria.
4. Turbo boost: con pocos núcleos activos la frecuencia es mayor.
5. SMT/Hyper-Threading: los hilos 5–8 comparten recursos físicos con los 1–4.
6. Costo del `lcg_skip` inicial ($O(\log N)$ por hilo).

3.5. III.4. Predicción con 64 núcleos

Con $f = 0{,}9884$ y $p = 64$:

$$S(64) = \frac{1}{0{,}0116 + 0{,}01544} = \frac{1}{0{,}02704} \approx 37{,}0.$$

Con 64 núcleos el speedup máximo sería $\sim$37×, lejos del ideal 64×. Aumentar p con N fijo reduce el tiempo pero **no mejora la precisión estadística** ($\sigma \propto 1/\sqrt{N}$). Aumentar N sí mejora la precisión: p compra *velocidad*, N compra *precisión*.

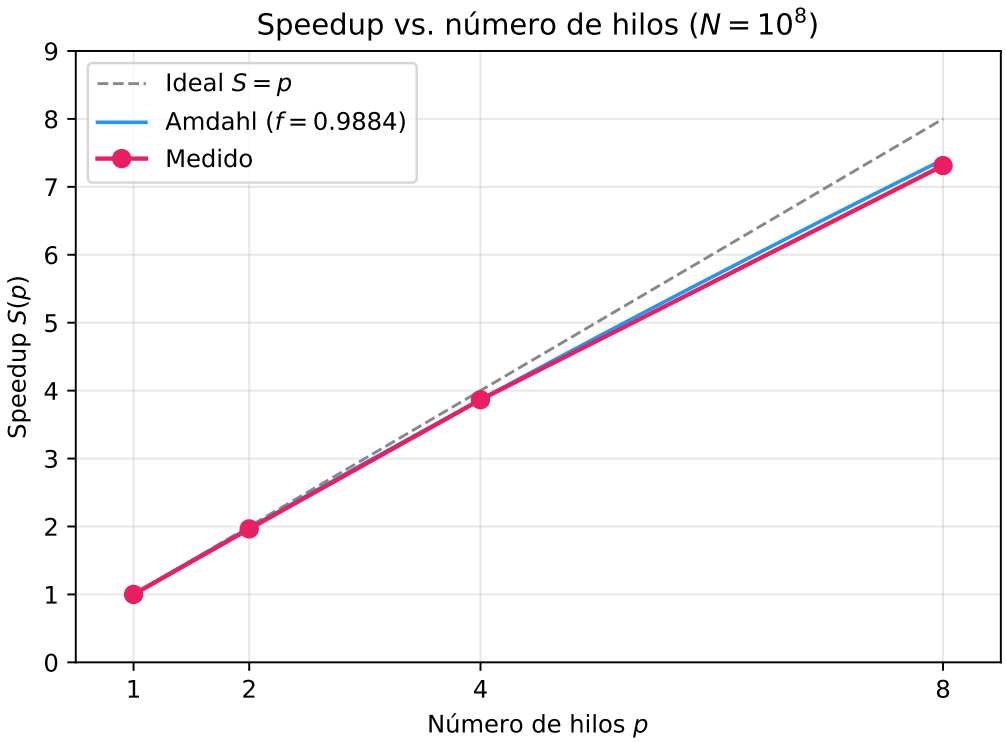


Figura 2: Speedup $S(p)$ medido, ideal y de Amdahl ($f = 0{,}9884$) en función de p.

3.6. III.5. Argumento de Gustafson

Gustafson observó que en la práctica los usuarios fijan el *tiempo de pared* y aumentan N con más procesadores. Su ley:

$$S'(p) = (1 - f') + f' \cdot p,$$

es lineal en p, sin techo asintótico. Para Monte Carlo, donde siempre hay apetito por más muestras, la visión de Gustafson es la relevante: más núcleos $\rightarrow$ más N en el mismo tiempo $\rightarrow$ más precisión.

A. Bitácora de uso de IA

Durante el desarrollo de este trabajo se utilizó un asistente de IA (Claude) como herramienta de *pair programming*. A continuación se detalla la división del trabajo.

Trabajo realizado por el estudiante

- Diseño e implementación del código secuencial de referencia (`montecarlo_referencia.cpp`), incluyendo la elección del generador LCG personalizado y la semilla.
- Implementación de la cabecera `rng_lcg.h` con el generador LCG y la función de salto `lcg_skip` basada en exponenciación rápida.
- Paralelización con OpenMP (`montecarlo_omp.cpp`): elección de las cláusulas `schedule(static)`, `reduction`, `firstprivate` y `private`; diseño del mecanismo de salto para reproducibilidad.
- Ejecución de todos los benchmarks: mediciones de tiempo para $p \in \{1, 2, 4, 8\}$ con 5 corridas cada una, y la comparación entre `schedule(static)` y `schedule(dynamic)`.
- Cálculos numéricos de speedup, eficiencia, fracción paralela f, métrica de Karp–Flatt.
- Generación de la gráfica de error vs. N con el script Python.
- Estructura general del informe y redacción de las secciones I.2, II.2, II.4, III.1 y III.4.

Trabajo asistido por IA

- Redacción y revisión de las secciones teóricas I.1 (demostración del orden $1/\sqrt{N}$, tabla RMSE) e I.3 (equivalencia del estimador paralelo).
- Desarrollo detallado de la sección II.3 (análisis de por qué compartir `rand()` entre hilos es incorrecto: data race y correlación por semilla idéntica).
- Análisis de la comparación static vs. dynamic (II.5): interpretación de los resultados del benchmark.
- Sección III.3 (comparación del speedup medido con ideal y Amdahl, identificación de los 6 factores que explican la brecha).
- Sección III.5 (argumento de Gustafson y contraste con Amdahl).
- Generación de la gráfica de speedup vs. p.

- Formateo LaTeX final, corrección de warnings de compilación y ajustes de presentación.

Metodología de trabajo

El flujo consistió en sesiones de pair programming donde el estudiante planteaba cada pregunta del parcial, aportaba los datos experimentales y las ideas centrales, y el asistente ayudaba a formalizar las respuestas, verificar cálculos y pulir la redacción. Todas las decisiones de diseño del código (elección del LCG, mecanismo de salto, cláusulas OpenMP) fueron del estudiante; la IA contribuyó principalmente en la parte analítica y expositiva del informe.